Lembar Kerja Praktikum P07 – Exception Handling

Lembar kerja ini menggantikan laporan praktikum. Isi **sambil** mengerjakan setiap langkah di modul, lalu kumpulkan di akhir sesi sebagai `LK-P07-<NIM>.pdf` bersama zip kode Anda.

Petunjuk pengisian

- Jawab singkat dan jelas. Satu sampai tiga kalimat per pertanyaan sudah cukup.
- Tempel output program sebagai **teks**, bukan tangkapan layar.
- Di L2 dan L3, isi kolom **prediksi** sebelum menjalankan program. Prediksi yang salah tidak mengurangi nilai; yang dinilai adalah penjelasan selisihnya.
- Total nilai LK 50 poin. Bagian L0 wajib diisi, tanpa poin.

L0 – Identitas dan parameter (wajib)

Isian	Jawaban
Nama	Muhammad Fharel
NIM	2511531010
Kelas	B
Tanggal praktikum	9 November 2026
Versi Java (baris pertama `java -version`)	java 27.0.1.0.3
Saldo awal e-wallet (`NimParams`)	Rp 250000
Stok awal KOPI-001	2
Stok awal RDG-002	2

L1 – Membaca crash dan menangani input (8 poin)

(a) Bedah stack trace `CrashDemo` skenario 2 dan 3 (4 poin). Contoh pengisian untuk skenario 1 ada di modul.

Yang dicari	Skenario 2	Skenario 3
Jenis exception	NumberFormatException	NullPointerException
Checked atau unchecked?	unchecked	unchecked
Pesan	For input string: "dua"	Cannot invoke "shop.model.Address.getCity()" because the return value of "shop.demo.CrashDemo$Customer.getAddress()" is null
Baris kode Anda yang pertama (file:baris, method)	CrashDemo.java:28, Method readQuantity	CrashDemo.java:32, method shippingCity
Dipanggil dari (file:baris, method)	CrashDemo.java:43, method main	CrashDemo.java:44, method main

Yang dicari	Skenario 2	Skenario 3
Apa penyebabnya, dengan kata-kata Anda sendiri?	gagal mengonversi string "dua" menjadi bilangan bulat (int) karena teks tersebut bukan format angka.	Objek alamat milik pelanggan (customer.getAddress()) bernilai null, tetapi method shippingCity langsung memanggil method .getCity() pada objek yang kosong tersebut.

(b) Jumlah `-2` (2 poin). Sebelum `ShopApp` diperbaiki, apa yang terjadi saat Anda memilih produk 2 lalu mengetik jumlah `-2`? Mengapa tidak muncul `NumberFormatException`? Apa beda "format input salah" dengan "aturan bisnis dilanggar"?

Jawaban: Input diterima tanpa crash. Perulangan for (int i = 0; i < quantity; i++) langsung berhenti karena 0 < -2 bernilai false, namun program tetap mencetak + -2 x Rendang Kemasan 250g dan lanjut meminta input berikutnya. Alasannya karena String "-2" merupakan format bilangan bulat bertanda yang sah, sehingga Integer.parseInt("-2") berhasil mengonversinya menjadi -2.

(c) `if` atau `catch`? (2 poin). Mengapa nomor produk di luar katalog dicegah dengan `if`, sedangkan teks yang bukan angka ditangkap dengan `catch (NumberFormatException _)`?

Jawaban: Nomor produk dicegah dengan if karena ukuran katalog sudah diketahui secara pasti sebelum diakses, sehingga pemeriksaan batas indeks (choice < 1 || choice > catalog.size()) dapat dilakukan secara murah dan langsung menggunakan percabangan biasa daripada menangkap IndexOutOfBoundsException. Sebaliknya, teks angka ditangkap dengan catch karena validitas format string dari input eksternal pengguna tidak dapat dipastikan sebelum proses parsing dijalankan.

L2 – Prediksi dulu, baru jalankan (12 poin)

Isi kolom **Prediksi** sebelum menjalankan program. Tulis baris-baris output sesuai urutan, misalnya `A1, A2, A4, A5`.

`FlowDemo` bagian A, B, C (7 poin: A 2, B 3, C 2)

Bagian	Prediksi output	Output sebenarnya	Cocok?	Penjelasan selisih (atau mengapa tebakan Anda benar)
A: `partA("12")` lalu `partA("12rb")`	partA(“12”) A1 mulai A2 jumlah = 12 A4 finally A5 lanjut partA(“12rb”) A1 mulai A3 bukan angka A4 finally A5 lanjut	== Bagian A == A1 mulai A2 jumlah = 12 A4 finally A5 lanjut A1 mulai A3 bukan angka A4 finally A5 lanjut	Cocok	Input "12" valid sehingga A2, finally (A4), dan A5 berjalan normal tanpa catch. Input "12rb" memicu NumberFormatException yang ditangkap catch (A3), lalu tetap menjalankan finally (A4) dan A5.
B: `partB(300000, "3")` lalu `partB(300000, "0")`	partB(300000, "3") B1 bulan = 3 B2 cicilan = 100000 B4 finally B5 hasil = 100000 partB(300000, "0") B1 bulan = 0 B4 finally B7 main menangkap: / by zero	== Bagian B == B1 bulan = 3 B2 cicilan = 100000 B4 finally B5 hasil = 100000 B1 bulan = 0 B4 finally B7 main menangkap: / by zero	Cocok	Pada "3", pembagian berhasil (B2) dan finally (B4) dieksekusi sebelum return ke main (B5). Pada "0", ArithmeticException tidak tertangkap di partB, sehingga finally (B4) tetap berjalan sebelum exception merambat ke main (B7).

Bagian	Prediksi output	Output sebenarnya	Cocok?	Penjelasan selisih (atau mengapa tebakan Anda benar)
C: `partC(5)`	partC(5) C1 cek stok C2 finally C3 sisa = 4	== Bagian C == C1 cek stok C2 finally C3 sisa = 4	Cocok	Ekspresi return dievaluasi setelah C1, namun finally (C2) tetap dijalankan terlebih dahulu sebelum method kembali ke main untuk mencetak C3.

`CariProdukDemo` bagian D (3 poin)

Perintah	Prediksi output	Output sebenarnya	Penjelasan: catch mana yang jalan, dan mengapa
`...` `CariProdukDemo 2`	D1 SKU: RDG-002 D5 pencarian selesai	D1 SKU: RDG-002 D5 pencarian selesai	Tidak ada catch yang jalan karena argumen "2" valid dan berhasil mengakses indeks 1 (RDG-002), lalu blok finally (D5) tetap dieksekusi.
`...` `CariProdukDemo abc`	D2 nomor harus bilangan bulat D5 pencarian selesai	D2 nomor harus bilangan bulat D5 pencarian selesai	NumberFormatException e (D2) jalan karena Integer.parseInt("abc") gagal mengonversi string non-angka (blok catch spesifik pertama yang cocok dari atas dipilih meski merupakan subclass IllegalArgumentException), lalu finally (D5) jalan.
`...` `CariProdukDemo 8`	D3 nomor di luar katalog: Index 7 out of bounds for length 3 D5 pencarian selesai	D3 nomor di luar katalog: Index 7 out of bounds for length 3 D5 pencarian selesai	ArrayIndexOutOfBoundsException e (D3) jalan karena indeks 7 (8 - 1) melebihi batas panjang array (3), lalu finally (D5) jalan.
`...` `CariProdukDemo` (tanpa argumen)	D4 Berikan satu nomor produk. D5 pencarian selesai	D4 Berikan satu nomor produk. D5 pencarian selesai	IllegalArgumentException e (D4) jalan karena kondisi argumen kosong memicu pelemparan IllegalArgumentException yang ditangkap oleh blok ketiga tersebut, lalu finally (D5) jalan.

L2.c – Ubah urutan catch (2 poin). Tempel pesan error compiler setelah catch `IllegalArgumentException` dipindah ke paling atas, lalu jelaskan artinya dalam satu kalimat.

Pesan error: error: exception NumberFormatException has already been caught

Artinya: Karena NumberFormatException merupakan subclass dari IllegalArgumentException, meletakkan catch (IllegalArgumentException e) di atas menyebabkan seluruh NumberFormatException sudah tertangkap oleh blok induk tersebut sehingga blok catch (NumberFormatException e) di bawahnya tidak akan pernah bisa dijangkau

L3 – Custom exception dan checkout (10 poin)

(a) Error berantai (2 poin). Setelah `PaymentMethod.pay` diubah menjadi `void ... throws PaymentFailedException`, berapa error yang muncul, dan di file mana saja? Mengapa satu perubahan di interface memengaruhi file-file itu?

Jumlah error: Muncul 4 error kompilasi File: EWalletPayment.java, BankTransferPayment.java, CreditCardPayment.java, dan CheckoutService.java.

Alasan: Mengubah tanda tangan method pay(long amount) pada interface PaymentMethod dari tipe kembalian boolean menjadi void dengan throws PaymentFailedException memaksa seluruh kelas implementasi serta pemanggilnya menyesuaikan tipe kembalian dan deklarasi exception-nya agar tetap memenuhi kontrak interface.

(b) Enam skenario `ScenarioRunner` (4 poin). Isi kolom prediksi dulu (pakai parameter L0), baru jalankan. Hasil diisi salah satu dari `[OK]`, `[SALDO]`, `[BAYAR]`, `[STOK]`, `[INVALID]`.

No.	Prediksi hasil	Prediksi sesudahnya: KOPI / RDG / TEH / saldo	Hasil sebenarnya	Sebenarnya: KOPI / RDG / TEH / saldo	Cocok?
S1 Kopi + Rendang, Reguler, e-wallet	[OK] E-Wallet DompetKu membayar Rp215000	KOPI=1 / RDG=1 / TEH=5 / saldo Rp35000	[OK] E-Wallet DompetKu membayar Rp215000	KOPI=1 / RDG=1 / TEH=5 / saldo Rp35000	Cocok
S2 Kopi x3, Reguler, transfer bank	[STOK] Stok KOPI-001 tidak cukup: diminta 1, tersedia 0	KOPI=1 / RDG=1 / TEH=5 / saldo Rp35000	[STOK] Stok KOPI-001 tidak cukup: diminta 1, tersedia 0	KOPI=1 / RDG=1 / TEH=5 / saldo Rp35000	Cocok
S3 Rendang x2, Reguler, e-wallet	[STOK] Stok RDG-002 tidak cukup: diminta 1, tersedia 0	KOPI=1 / RDG=1 / TEH=5 / saldo Rp35000	[STOK] Stok RDG-002 tidak cukup: diminta 1, tersedia 0	KOPI=1 / RDG=1 / TEH=5 / saldo Rp35000	Cocok
S4 Kopi, Ekspres, kartu kredit	[SALDO] Saldo tidak cukup: perlu Rp103000, tersedia Rp100000 (kurang Rp3000)	KOPI=1 / RDG=1 / TEH=5 / saldo Rp35000	[SALDO] Saldo tidak cukup: perlu Rp103000, tersedia Rp100000 (kurang Rp3000)	KOPI=1 / RDG=1 / TEH=5 / saldo Rp35000	Cocok
S5 Keranjang kosong	[INVALID] IllegalStateException: Cart is empty	KOPI=1 / RDG=1 / TEH=5 / saldo Rp35000	[INVALID] IllegalStateException: Cart is empty	KOPI=1 / RDG=1 / TEH=5 / saldo Rp35000	Cocok
S6 Teh x2, Reguler, e-wallet	[SALDO] Saldo tidak cukup: perlu Rp80000, tersedia Rp35000 (kurang Rp45000)	KOPI=1 / RDG=1 / TEH=5 / saldo Rp35000	[SALDO] Saldo tidak cukup: perlu Rp80000, tersedia Rp35000 (kurang Rp45000)	KOPI=1 / RDG=1 / TEH=5 / saldo Rp35000	Cocok

Pilih satu skenario yang prediksinya meleset (atau, jika semua cocok, skenario yang paling menarik), lalu jelaskan mengapa hasilnya seperti itu. Sebut juga mengapa pesan `[STOK]` menyebut "diminta 1".

Jawaban: skenario yang paling menarik adalah Skenario S2. Meskipun S2 memakai metode transfer bank, proses checkout gagal lebih awal pada pengecekan stok ([STOK]) sebelum menyentuh tahap pembayaran karena sisa stok KOPI-001 setelah S1 hanya tinggal 1. Saat reservasi berjalan, unit pertama berhasil dipesan, lalu unit kedua gagal dan melempar OutOfStockException; blok catch kemudian mengembalikan 1 stok yang sempat dipesan (failure atomicity) sehingga stok akhir tetap KOPI=1. Pesan [STOK] menyebut "diminta 1" (bukan "diminta 3") karena pesanan Kopi x3 dicatat ke dalam Order sebagai 3 baris item (Order.Line) terpisah yang masing-masing berjumlah 1, dan CheckoutService memesan stok secara berurutan per baris (inventory.reserve(sku, 1)). Baris pertama berhasil mengambil 1 stok yang tersisa, sehingga saat baris kedua meminta 1 unit lagi, stok yang tersedia sudah 0 (diminta 1, tersedia 0).

(c) Mengembalikan stok lalu `throw e` (2 poin). Di method `checkout`, mengapa blok catch mengembalikan stok lalu melempar ulang exception yang sama, bukan mencetak pesan untuk pembeli? Apa yang terjadi pada stok jika `release` dihapus?

Jawaban: Blok catch mengembalikan stok untuk menjaga failure atomicity dan melempar ulang throw e karena CheckoutService berada di lapisan logika bisnis yang tidak bertugas menampilkan output ke pengguna, penyampaian pesan diserahkan kepada pemanggil di lapisan aplikasi (ShopApp.buy). Jika pemanggilan release dihapus, stok barang yang sempat berhasil dipesan sebelum kegagalan terjadi akan tetap berkurang secara permanen meskipun transaksi batal.

(d) Urutan catch di `buy` (2 poin). Mengapa catch `InsufficientBalanceException` harus ditulis di atas catch `PaymentFailedException`? Apa yang terjadi jika urutannya ditukar?

Jawaban: InsufficientBalanceException adalah subclass dari PaymentFailedException, dan Java memeriksa blok catch secara berurutan dari atas ke bawah sehingga exception yang lebih spesifik harus ditulis lebih dulu agar mendapat penanganan khusus. Jika urutannya ditukar, seluruh InsufficientBalanceException akan langsung tertangkap oleh catch PaymentFailedException di atasnya sehingga blok catch InsufficientBalanceException di bawahnya menjadi tidak pernah terjangkau dan memicu compile error (exception InsufficientBalanceException has already been caught).

L4 – Katalog dari file (6 poin)

(a) Baris katalog yang dilewati (3 poin).

Baris ke-	Isi baris	Apa masalahnya?	Exception yang tertangkap (nama class)
Baris 3	KUE-003; Kue Lapis Legit;95rb; 800	Format harga bukan angka bulat yang valid ("95rb" mengandung huruf "rb")	NumberFormatException (ditangkap sebagai IllegalArgumentException)
Baris 5	BATIK-005; Kain Batik Tulis	Jumlah kolom kurang (hanya ditemukan 2 kolom, seharusnya 4 kolom)	IllegalArgumentException (expected 4 fields, found 2)
Baris 6	SAMB-006; Sambal Lado Mudo 200g;-45000;250	Nilai harga bernilai negatif (-45000), melanggar aturan bisnis harga produk	IllegalArgumentException (Price must not be negative)

(b) Ketika file tidak ada (3 poin). Tempel output `java -cp bin shop.app.ShopApp katalog-cadangan.csv`. Apa isi `e.getMessage()` dan `e.getCause()`? Kalau `ShopApp` hanya mencetak `e.getCause().getMessage()`, apa yang dilihat pengguna, dan mengapa itu kurang membantu?

Output:
Toko tidak bisa dibuka: Katalog gagal dibaca: katalog-cadangan.csv
Penyebab asli: java.nio.file.NoSuchFileException: katalog-cadangan.csv

Jawaban: e.getMessage() berisi "Katalog gagal dibaca: katalog-cadangan.csv" (pesan tingkat bisnis), sedangkan e.getCause() berisi java.nio.file.NoSuchFileException: katalog-cadangan.csv (penyebab teknis asli). Jika hanya mencetak e.getCause().getMessage(), pengguna hanya akan melihat teks "katalog-cadangan.csv" saja tanpa konteks bahwa file tersebut tidak ditemukan atau gagal dimuat sebagai katalog.

L5 – Code review FragileShop (10 poin)

Tabel temuan (8 poin, 2 per method).

Method	Apa masalahnya?	Dampaknya (bagi pembeli atau programmer)	Perbaikan
`payOrder`	Menelan exception secara diam-diam (exception swallowing) dengan blok catch (PaymentFailedException _) {} yang kosong.	Saat pembayaran gagal (saldo kurang), error hilang tanpa jejak dan program tetap berjalan seolah pembayaran berhasil.	Hapus try-catch kosong, tambahkan throws PaymentFailedException, dan tangani kegagalannya di main.
`isInStock`	Menggunakan mekanisme exception (reserve lalu release) hanya untuk menjawab pertanyaan boolean ya/tidak.	Menimbulkan overhead pembuatan stack trace untuk kondisi normal serta mutasi stok sementara yang tidak perlu dan berisiko menimbulkan race condition.	Periksa ketersediaan stok secara langsung tanpa exception: return inventory.available(sku) > 0;
`readCatalogHeader`	Membuang penyebab asli (cause) saat membungkus IOException ke dalam CatalogLoadException dengan meneruskan null.	Programmer kehilangan informasi diagnostik akar masalah teknis (e.getCause() bernilai null) saat terjadi kegagalan I/O.	Sertakan penyebab aslinya sebagai argumen kedua: throw new CatalogLoadException("Katalog gagal dibaca: " + file, e);
`addToCart`	Menggabungkan (multi-catch) kesalahan input pembeli (IllegalArgumentException) dan stok habis (OutOfStockException) serta menampilkan e.getMessage() teknis mentah ke pembeli.	Pembeli melihat pesan error teknis Java (For input string: "dua") yang membingungkan dan tidak ada pemisahan penanganan antara salah ketik dan stok habis	Pisahkan blok catch: berikan pesan validasi yang ramah pembeli untuk kesalahan format/jumlah input, dan pesan khusus informasi stok untuk OutOfStockException.

Output `FragileShop` setelah `payOrder` dan `readCatalogHeader` diperbaiki (2 poin).

```
Output:
1) Pembayaran gagal: Saldo tidak cukup: perlu Rp85000, tersedia Rp50000
2) Kopi tersedia? true
3) Gagal: For input string: "dua"
   Gagal: Stok KOPI-001 tidak cukup: diminta 5, tersedia 1
4) Katalog gagal dibaca: katalog-lama.csv | penyebab: java.nio.file.NoSuchFileException: katalog-lama.csv
```

Penutup

Hasil SelfCheck terakhir. Salin dua baris berikut dari output SelfCheck.

```
=== HASIL: ...
=== HASIL: 24 / 24 pemeriksaan lulus (nilai SelfCheck 50 / 50) ===
```

```
=== TANTANGAN: ...
Tidak ditampilkan
```

Refleksi (4 poin).

1. (2 poin) Mengapa kesalahan ketik pembeli (misalnya `dua` sebagai jumlah) perlu ditangani berbeda dari stok yang habis? Hubungkan dengan pilihan checked/unchecked.

Jawaban: Kesalahan ketik seperti "dua" merupakan kegagalan format input yang menggunakan unchecked exception (NumberFormatException / IllegalArgumentException) dan cukup ditangani di lapisan input agar pembeli mengulang ketikannya. Sebaliknya, stok habis adalah kegagalan aturan bisnis saat transaksi yang menggunakan checked exception (OutOfStockException), sehingga compiler memaksa lapisan pemanggil menanganinya secara eksplisit

2. (1 poin) Apakah pesan exception teknis, misalnya `For input string: "dua"`, sebaiknya ditampilkan langsung kepada pembeli? Beri satu contoh dari praktikum ini.

Jawaban: Tidak, karena pesan teknis membingungkan pembeli awam serta dapat membocorkan detail internal program, sehingga perlu diterjemahkan menjadi instruksi yang mudah dipahami. Contohnya pada FragileShop.addToCart (L5), pesan mentah For input string: "dua" kurang ramah bagi pembeli dibandingkan ShopApp (L1) yang menangkap NumberFormatException lalu menampilkan pesan jelas seperti "Jumlah harus berupa angka.".

3. (1 poin, dinilai penuh asal diisi jujur) Satu hal tentang exception yang masih membingungkan atau ingin Anda tanyakan di pertemuan berikutnya.

Jawaban: Kapan sebaiknya kita membuat custom exception sendiri dibandingkan cukup memakai exception bawaan Java?

LINK REPOSITORY GITHUB:
https://github.com/MuhammadFharel/PrakPBO_B_2511531010/tree/after-uts/prakPBO_2026_B_2511531010